

Global Facilities - Design & Construction Standard - I&C - SCADA

ECN Facility:	GLOBAL FACILITIES	ECN Area:	ENGINEERING
Owner(s)	ENGINEERING	Target Audience:	All Facilities
Document Type:	Standard		
		Document ID (pdf):	A3YRXSD74VDV-57553043-486
Version:	1		
Current ECN:	101115830	Last Modified:	03/21/2022

[ECN History](#)[Submit a Change Request](#)

1.0 Purpose

This standard defines the minimum requirements for design, supply, and installation of SCADA and Remote Operations Software that are part of Unified Operations.

2.0 Scope

2.1 Site(s) impacted

This document applies to all Micron Facilities employees, contractors, and vendors at all Micron sites for new Greenfield FAB Construction projects.

2.2 Vendor Scope

2.2.1 The following, not limited to, are in the scope of SCADA system integrator:

- a. System Architecture drawing (PLC, SCADA, Network Switches, Third Party Interface, etc).
- b. TCP/IP communication interface list.
- c. Hardware list.
- d. Software list.
- e. SCADA tag list.
- f. Alarm list.

- g. Graphic List and Navigation.
- h. Trend list and configuration.
- i. Third party interface list.
- j. Inspection and Test Plan (ITP).
- k. Testing and Commissioning.
- l. System Training.
- m. As-Built Document.
- n. Operation & Maintenance Manual.
- o. 24/7 Monitoring of system at control room from system commissioning to handover of the system to Micron.

3.0 Definitions and Acronyms

- **A/D** – Analog to Digital
- **AC** – Alternating Current
- **AI** – Analog Input
- **AO** – Analog Output
- **D/A** – Digital to Analog
- **DC** – Direct Current
- **DI** – Discrete Input
- **DO** – Discrete Output
- **FAT** – Factory Acceptance Test
- **FMCS** – Facility Management and Control System
- **FCT** – Facilities Central Team. This is used interchangeably with Global Facilities Engineering Team and associated to O&M BKMs
- **GFTT** – Global Facilities Technology Team. This is used interchangeably with Global Facilities Engineering Team and associated to design standards
- **GFC** – Global Facilities & Construction Team. This is used interchangeably with Global Facilities (GFac), Facilities Central Group (FCG), World Wide Operations (WWOps), World Wide Facilities (WW_FACILITIES), Corp Facilities
- **Global Facilities Central Team** – This is used interchangeably with Innovation & Integration Team
- **Global Facilities Construction Team** – This is used interchangeably with Global Construction Team
- **Global Facilities Operations Support Team** – This is used interchangeably with Global Facilities Operations Team
- **Global Facilities Planning Team** – This is used interchangeably with Global Facilities Line Planning Team
- **I&C** – Instrumentation and Control
- **IT** – Information Technology
- **O&M** – Operation and Maintenance
- **OPC** – Object linking and embedding for Process Control

- **OT** – Operational Technology
- **PID** – Proportional Integral and Derivative control algorithm
- **PLC** – Programmable Logic Controller
- **RIO** – Remote I/O
- **SAT** – Site Acceptance Test
- **SCADA** – Supervisory Control and Data Acquisition
- **SI** – System Integrator

4.0 Roles and Responsibilities

Roles	Responsibilities
GFTT Lead (Discipline Engineer)	• Develop and maintain System Standard document • Ensure Best Known Methods (BKM), lessons learned, and emerging technologies are documented in System Standard • Perform periodic document review and update System Standard
FCT Lead (Discipline Engineer)	• Review and provide technical feedback on System Standard • Support the Site teams by providing knowledge and technical expertise
Site Project Construction Teams	• Review and provide technical feedback on System Standard • Incorporate System Standard into Project Engineering, Design, and Construction • Document project deviations, exceptions, and exclusions from Design Standard • Share new lessons learned to GFTT Lead, to determine if it is Best Known Method (BKM) that can be captured in future revision of Design Standard
Site Facilities Teams	• Use System Standard as technical reference

5.0 Applicable Codes and Standards

The following industry, association, codes and standards shall be followed as applicable to the design, fabrication, assembly, and testing of all equipment furnished under this specification. Deviations from this must be agreed formally by Micron.

- a. IEEE 1985, Guidelines for the Documentation of Software in Industrial Computer Systems.
- b. Instrument Society of America (ISA)
- c. Good Automated Manufacturing Practices (GAMP)
- d. ISO 9001 – Quality Management Systems
- e. Underwriters Laboratories (UL): UL 508 and UL 508A.

6.0 System Description

- 6.1 The SCADA is comprised of data collection, data processing, and communication servers connected by an Ethernet network.
- 6.2 In general, based on complexity of the facility system being monitored, the SCADA / HMI shall incorporate graphical representation of systems to display status and alarms. Graphics shall be representative of, but are not limited to, Piping and Instrument Diagrams (P&ID's), Process Diagrams (PFD's), I/O list, wiring schematics and/or instrument loop diagrams.
- 6.3 The SCADA will run on multi-user, multitasking operating system using remote terminals and/or personal computers as operator interfaces.

7.0 General Requirement

The software platform shall consist of an edge to enterprise visibility, human machine interface (HMI) with supervisory and process control, real-time data acquisition, alarm and event management, historical data collection, report generation, analytics, dashboarding, modeling, local or indirect remote communications to PLC's, RTUs, equipment, third party monitoring system, and IIOT Devices. The software should be scalable, fully redundant, provides industry standard protocols, IIOT, ease of managing global standards and integration to remote operations across the sites/facility and enable connected worker capability.

The software should be hardware and software interoperable to enable connectivity with other IT Systems like Asset Management, Enterprise Software or Design Software and enable remote operations capability. The software should enable IT-OT Convergence and provide the system easier to connect, visualize and add context to information. The software shall be easy-to-use, deployable at edge, on-premise or cloud supported. This system should have a global standard management environment to define assets, its visualization and context which is contextually aware, responsive and shall have an open architecture, extensibility to build apps, drivers and supported on the latest Microsoft Windows Server and Client operating systems. The system should be accessible from other non-Windows Devices.

The system shall have the built-in flexibility to permit easy configuration in accordance with the specific end user requirements, as well as quick and easy modification by the end user in the field.

The software shall consist of a suite of off-the-shelf modular components from a single software manufacturer that are tightly integrated together to perform all operation control and visibility system functions. The suite shall contain an HMI for process visualization, dashboards, a real-time relational database for historical data collection with an open and accessible query interface, client tools for trending and reporting both within the HMI and as standalone utilities. The system should have capability to provide edge to enterprise visibility like KPI, Dashboarding, Alarm and Events, Alerts and notification and on-premise, hybrid and cloud deployment. Communication drivers for IOT, PLC's, PACs and RTU's shall be provided and supported by the software vendor.

The system shall be scalable so that a small stand-alone application can easily be joined to or expanded into a large, distributed control network supported by redundant application servers, database servers, and single or redundant communication servers providing information to multiple workstation clients. The software shall also be able to easily interface with external databases, modeling and simulation, Enterprise Asset Management (EAM), Condition Based Monitoring (CBM), Maintenance Management Systems (CMMS), ERP, LIMS, Cameras, Workflow, Data Lake and GIS systems.

7.1 Open Approach to Software.

Using common interfaces and commercial software where appropriate, the software shall be able to communicate and interact with most common hardware and software components of an industrial enterprise, including but not limited to, field I/O, IIOT, databases and enterprise business systems.

7.2 Extensibility

Recognizing that the specific functionality of the supplied software is unlikely to meet 100% of the requirements for the SCADA system, the software provided shall be extensible by a competent system integrator or systems engineer.

No special programming experience shall be required, although it is permissible and indeed desirable that advanced toolkits be available and supported to enable custom software components to be integrated with the SCADA software.

The software architecture shall be capable of supporting up to Two million (2,000,000) I/O and one hundred (100) local nodes in a distributed network.

7.3 Standards

In conjunction with the Open Approach to the software, the software shall also support the use of industry standards where applicable.

The software shall also promote and encourage the use of standards throughout software design as a best practice.

The software should provide enterprise-wide standards compliance across processes, functional teams, and sites. Its common infrastructure adds universal context to real-time processes, alarms, events, and archived historical data across disparate business systems – creating a single, common information model that makes system design and maintenance more efficient, flexible, and secure.

7.4 Operating System Support

The software specified shall be licensed to support the latest Microsoft Operating System and Server database versions available in the market, as well as capability to have backward compatibility to at least one or two versions.

In case of a later release of operating system the vendor shall have a defined statement and roadmap for support of that operating system. The software vendor shall provide an up-to-date list of Microsoft Operating System updates and/or patches that are approved for release on software solution.

7.4.1 Virtualization System Support

The SCADA software shall also be supported in virtual machine environment using VMware (vSphere and vMotion), Microsoft Hyper-V, Cloud Virtualization (such as Azure) or Nutanix Hyper Advisor environments.

The software vendor shall ensure that virtual environments support architectures with High Availability (HA) and Disaster Recovery (DR) configurations and provide Cloud integration. Hardware support shall include multi-core and multi-processor computers.

8.0 Development Environment Software Requirements

This section describes the engineering development requirements of all system software functions in any combination as follows:

8.1 Multi-User Development Environment

The Development Environment shall provide simultaneous multi-user capabilities, where users are subject to security permissions based on individual system-wide roles.

The software platform should ensure application integrity through corporate standardization compliance across processes, functional teams, and sites. The entire system configuration, administration and deployment shall be managed locally or remotely. It should enable rapid change management to align with changing business or technology drivers, while automatically propagating changes systematically across projects reduces risk and the time.

The new visualization configuration should support concurrent development by enabling multiple developers to work concurrently and remotely on the same application at the same time - without any lock-out issues.

8.2 Standards/Object Model/ Templatized Model

The Development Environment shall utilize a standard asset model to allow designing an application which models the physical characteristics of the SCADA/Unified Operations system, including geographical topology, physical equipment, asset details and computer locations. The system shall use the concept of Application Objects. These Objects may represent real world devices such as computers, PID loops, Motors, Pumps, Valves, etc., or informational objects such as external database readers and writers, XML readers and writers, etc.

Application objects shall closely model the physical representation of plant equipment and devices and not be bound to a "tag-only" topology. This shall include the ability to create complex, multi-variable data structures.

8.2.1 Development Environment Views

The Development Environment shall have the ability to

- Organize Object Templates into user configurable Template Toolboxes.
- Organize visualization graphics into user configurable Graphic Template Toolboxes.
- View and configure the application from a plant model perspective.
- View and configure the application from an Application Object deployment perspective.
- View and configure objects showing the genealogy of the object from the object back to its parent template(s) and back to the base object, no matter the length of the parent to child relationship.
- View and configure the application from an IO device connectivity perspective.

8.2.2 User Security

The Development Environment shall be capable of utilizing Microsoft operating system security, for example Active Directory Domains and groups, to allow users access to view, configure, or modify templates and Application Objects.

The application should be able to manage and configure different user groups with segregations of functionalities to access, view, configure or modify application objects. The users/groups authentication mode should leverage windows Active directory domains.

8.2.3 Development Audit Trail

The Development Environment shall provide an audit trail of the check-out, check-in and revision history for each template or application object and shall include the user ID of the person making the change(s), the time and date stamp, and a detailed summary of the changes made.

8.2.4 Centralized Configuration of System Wide Settings

The Development Environment shall allow centralized management and configuration of system wide settings including the following:

- Configuring standard graphical representation of quality and status.
- Configuring standard styles for graphical elements
- Configuring standard styles for numeric formatting.
- Organization of alarms in four severity categories: critical, high, medium, and low for easy interpretation by operators.

- Mapping alarm priorities to alarm severities.
- Selecting alarm severities that can be shelved.
- Selecting icons to represent each of the supported alarm severities.
- Selection of alarm severities to be historized.
- Selection of event types to be historized.
- Configuration of plant state-based alarming.
- Configure Style Library which sets visual and numeric styles to View Applications.

8.3 Object and Code Re-Use

The Development Environment shall promote code re-use by using templates to define application objects. Templates may be customized to create new application object templates and maintain parent-child relationships of the object definitions.

Templates are high-level definitions of the devices in application environment. User can define a template for an object, like a valve, one time and then use that template when it is needed to define another instance of that item.

A template shall specify application logic, alarms, security, and historical data for an object. A template shall also define an area of environment. You can extend and customize a template by adding attributes, scripts, or features to meet the specific needs of your environment.

Objects inherit attributes, scripts and features from their parents. Application Server comes with predefined templates, called base templates. All templates you create are derived from base templates.

8.4 Object Repository

The Development Environment shall utilize a centralized repository for templates and application objects, object hierarchy, deployment configuration, and genealogy. It shall also provide the option to utilize the same repository for storage and management of graphical templates and the visualization application.

8.4.1 Configuration Integrity

The Development Environment shall allow users with configuration rights to view objects and ensure that only one person can check out for change a specific template or application object at any one time.

8.5 Object Templates

The Development Environment shall provide a mechanism to develop Application Object Templates. These object templates shall be used to create the individual instances of the objects that perform the SCADA tasks. Object Templates shall be able to contain other object templates in a hierarchical (parent/child) relationship. A child template shall inherit attributes and characteristics of its parent, but permit these inherited features to be overridden or customized in a controlled manner. Objects may contain general object configuration, input/output definitions, internal attribute definitions, internal documentation for configuration help, user defined attribute definitions, alarm definition, history definition, security and contained scripts. The Application Object shall be able to host an associated graphic or graphics for representation in visualization.

8.5.1 Standard Object Templates

The Development Environment shall expose Base Templates, defined as those provided by the software manufacturer and User Defined Templates. An Object Toolkit shall be available to allow the user to create new base objects using Microsoft Visual Studio languages such as Microsoft Visual C++ and Microsoft Visual C#.

8.5.2 Object Security

The Development systems shall be able to configure security within the model. The security system shall support an object based hierarchical model. This model shall allow for the creation of Security Groups that contain assigned Application Objects. The model shall allow for the creation of Operator Roles that can be assigned to Security Groups. Operator Roles shall allow for the assignment of configuration database permissions, and for runtime operational permissions, and access to visualization of certain windows. At a minimum, runtime operational permissions shall allow for:

- The access or denial of the ability to acknowledge an alarm in the runtime environment
- The modification of configuration attributes which allows users to configure the attribute's value (for example, a PLC register that defines a discrete input).
- The modification of operational attributes which allows users with operational permissions to do
- Certain normal day-to-day tasks like changing setpoint, output and control mode for a PID object, or commanding a Device object.
- To open and view a process or application window.
- The modification of tune attributes which allows users to tune the attribute in the runtime environment. Examples

of tuning are attributes that adjust alarm setpoints and PID sensitivity.

- The ability to Shelve Alarms.
- Users assigned to a particular role shall inherit all parameters that were assigned to that Role and Security Group.

8.5.3 Object Protection

With the purpose of helping Design, Distribute and protect standards, the development system shall allow exporting object templates in a protected form, so that others can make use of them without being able to modify them. The same shall apply to graphical templates.

8.5.4 Integrated Historical Data Configuration

The Object Template shall allow the configuration of attribute historical data storage within the object using a simple checkbox. No separate tool shall be required.

8.5.5 Integrated Alarm Data Configuration

The Object Template shall allow configuration of a connection to an alarm sub-system that supports condition-oriented alarms (LoLo, Lo, Hi, HiHi, Rate-of-Change Deviation, etc.), event- oriented alarms (True/False, Fail to Open, Fail to Close, Command Disagree, etc.) with predefined tools that will allow the developer through the process of defining the configuration.

8.5.6 Integrated Graphics

The Object Template shall allow association and configuration of one or more graphical representations for visualization purposes. The visualization application itself must be able to be represented and contained by an Object Template.

The object template shall allow the developer to create many different graphical representations that are tightly associated with the attributes of the object. As an example, there may be several styles of graphical depictions for a pump, from a classic ISA graphic to a situational awareness graphic. The developer then has the choice of which needs to be contained in the Runtime environment, or even to potentially switch between styles. The Runtime Application is also managed and contained within a Template.

8.5.7 Object Template Application Logic Scripts

The Object Template shall allow association and configuration of one or more logic scripts. Scripts contained by an Object Template shall utilize Microsoft .NET (dot NET) technology and compile to the .NET Common Language Runtime. The scripting shall be easy to program using English-like statements and shall not require any knowledge of any other programming logic. The user shall be able to edit or modify the logic scripts while the system is monitoring the process. The scripting language shall include selection boxes and pull-down menus to permit statements to be created without having to type tag names or specific commands. The software shall support basic commands and operations such as IF, THEN, ELSE, ELSE IF, AND, OR, NOT, ADD, SUBTRACT, MULTIPLY, DIVIDE, EQUAL TO, NOT EQUAL TO, GREATER THAN, and LESS THAN.

The scripting editor shall provide auto-complete capabilities. There shall be a full library of more complex math and system script functions available. A validate button shall be available to ensure proper syntax and provide indication of errors to eliminate any problems at run-time. The scripting language shall support the use of inline Microsoft .NET script function calls.

On-line help for each script function shall include actual working examples that can be copied and pasted into the script editor. Dot NET functions provided by Microsoft shall utilize documentation provided by Microsoft.

System logic shall support configuration of objects to automatically perform functions such as increase setpoints, perform totalization and check the status of process parameters to take appropriate action.

8.5.8 Status Monitoring and Alarming

System Logic shall support the configuration of objects to monitor the status of any tag name or hierarchical name attribute in the system and perform specific functions based on the type and status of an alarm condition.

8.5.9 Application Control and Data Change Logic

The system shall support the configuration of objects that perform application control to change the state of discrete points, show windows, download recipes, etc. This application logic shall be able to also start and stop other application programs such as Microsoft Excel, Microsoft Word, Crystal Reports, Dream Reports and other Windows based applications.

The system shall be able to configure scripts to be executed when an instantiated object starts up, when the instantiated object goes on scan, on a condition on true, while true, on false and while false, periodically, when the instantiated object goes off scan, and when an instantiated object shuts down.

8.5.10 Conditional Control and Data Change Logic

The system shall support the configuration of objects that perform application control based upon a user definable state of an object and attribute or upon the result of an expression involving multiple object tag names or hierarchical names, including discrete object tag names or hierarchical names on/off state, alarm states such as Lo, LoLo, Hi, HiHi, or equivalence to a specific value.

It shall be possible to define Condition Logic scripts that execute once when the condition expression becomes true, once when the condition expression becomes false or while the condition is true or while the condition is false at a user definable rate of 50 milliseconds minimum, or as an event-based script which executes when a value changes. The alias name used for navigation is a full attribute that can be referenced in runtime.

8.5.11 Configuration of I/O Objects

The Development Environment shall provide a communication manager of I/O Server Application Objects to provide remote data server installation, configuration activation and operations, and extensive protocol diagnostic troubleshooting.

The Development Environment shall provide the ability to automatically assign application objects to devices and of specifying input sources and output destinations on attributes.

8.5.12 Object Template Derivation, Object Instantiation and Inheritance

It shall be possible to derive new templates from existing templates, whether Base (i.e. supplied by the software manufacturer) or user defined. A derived template shall inherit the entire configuration of the parent object when generating a new template instance. Templates may contain other templates in a hierarchy. Derived templates shall maintain any hierarchy that the parent template contains.

The Development Environment shall be able to lock specific attributes to allow changes to the parent template to pass through to the new instance and all children of the new instance. Locked Attributes shall not be modifiable in the child template.

8.5.13 Object Deployment

The Development Environment shall utilize the concept of deployment. Deployment shall be defined as the remote installation of any Application Object, its children, dependencies and any other software required and bound by the Application Object for the Application Object to successfully operate, including but not limited to .NET Controls and Dynamic Link Libraries (DLLs).

All instantiated Application Object components shall be configured and deployed from the Development Environment to target workstations and servers.

8.5.14 Status of Objects

The Development Environment shall provide visual feedback as to the deployed status of any Application Object.

The Development Environment shall provide visual feedback as to the status of any Application Object that has a change pending.

8.6 Import/Export Utility

The Development Environment shall support import and export of the application model into a human readable file format such as CSV (comma separated file format) for editing in a spreadsheet application such as Microsoft Excel.

It shall be possible to instantiate templates and application objects from the CSV load by only populating the appropriate columns in the spreadsheet that are required for the instantiation/configuration of the desired objects.

8.6.1 Application Conversion

The SCADA vendor must provide support to convert other vendor applications, such as FactoryTalk ME/SE, PanelMate or PanelBuilder32.

8.7 HMI or Enterprise Visualization Development Software Requirements

This section describes the engineering development requirements of all HMI system software functions. This includes development of color graphic displays, configuration of the real-time and historical database, alarms, I/O communications to field devices and application setup of clients and servers on the SCADA network.

The system should provide high performance engine can render 50 graphic displays of 4K resolutions per application – 6 to 10x faster in graphic rendering performance which can scale from edge to enterprise visualization. The system 1/10 memory footprint (e.g. 200 clients on single terminal server). The system should have scalability

and run an entire 100K IO application project on a single machine with specification provided.

Key benefits and capabilities should include:

- Delivering more simplicity- HMI runtime pan and zoom, multi-touch gestures.
- Increasing productivity- Ease to use and debug script editor.
- Improving performance- advanced events and alarms management, change propagation.
- Saving development time – Application templates, change propagation, object and symbol wizard, global styles, situational awareness library, commonly used controls and interface.
- Improving engineering productivity- Repeatable window templates.
- Leveraging the power of Graphics with 'Frame' type windows – Ease of defining layout, dynamic binding in runtime, web HMI
- Multi-monitor layout – System should provide capability to support multi-monitor capability with zero scripting
- Ability to add a background image and/or color that will span an entire Layout.
- Designing applications for any target resolution on any device.
- Advanced dashboarding, reporting and cloud integration – seamless reporting, dashboarding, cloud integration
- Create and manage multiple Galaxy Style Libraries and switch between them at runtime.
- Capability to import web widgets and place them on panes and allow interaction with properties. Subscription-based Platform Communications Platform subscriptions are dynamically built as required to minimize inter-platform communication.

All development and configuration shall be persisted in one or more common file or database repositories that provide a single point of configuration.

The development environment shall provide the ability to host and manage the visualization application in a common repository. Furthermore, there shall be a common naming convention for objects and tag names that is enforced by the development tools.

In addition to native application support on the above systems, the same applications shall be configured to run in thin client sessions or remote desktop sessions hosted by disk or diskless PC's served by Microsoft Windows 2012 R2 or later Remote Desktop Services (RDS - formerly known as "Terminal Services").

8.7.1 Internationalization Considerations

- It shall be possible, using the SCADA system development software, to construct an application which at runtime may be dynamically switched from one language to another.
- The system should support any currently available language setting of the operating system. Multiple languages (more than two) shall be able to be configured.
- Strings used in text objects with graphics shall be able to be configured to be displayed in multiple languages, to be dynamically set by the system on operator log in based on the locale preferences of the operator.
- Alarm messages to be configured and displayed in the currently selected language.
- Text with no equivalent translated string in the current language shall be displayed in the default language.

8.7.2 Graphics Display Development

The SCADA system software shall include an object-oriented color graphics display generator with full animation capabilities to provide users with a realistic and efficient visualization of the SCADA system process. It shall provide graphical capabilities to allow design of highly efficient HMI applications aimed at helping operators to easily achieve a state of situational awareness in relation to the process.

- All graphical editing operations shall be point and click selecting icons from floating and docking tool bars, pull down menus or keyboard commands.
- The graphics development environment shall include tools to assist in efficient graphic design by displaying estimated runtime graphical performance statistics. It shall be possible to perform a functional test of any graphic display by switching to the runtime mode with a single mouse click.
- The development environment client tool shall be capable of running in a Remote Desktop or Terminal Services Session.
- The display editor shall include the tools detailed in this subsection for display drawing, linking and animation.
- The graphics editor shall include a broad library of out-of-the-box complex graphical objects and process symbols such as meters, pushbuttons, sliders, gauges, pumps, motors, tanks, valves, trends, alarms, and controller faceplates. All complex objects shall be scalable to any size and may include animation links to

provide dynamic response based on real-time data or user action.

- The application should enable the visual representation of data using symbols or graphics which can be easily repeated. The visual platform should enable repeat multiple times with no coding or scripting and map data. The graphics can also be sorted into lists or grids. The editor shall include the ability to develop reusable configurable library symbols without the need for a tool kit nor software development tools.
- The development workstation shall be able to use any of the operating systems of **Section 7.4**.
- Graphical Objects

Graphics shall be based on object technology as Application Objects. Graphical objects shall be able to contain other graphical objects and shall be able to expose and manipulate attributes and properties.

a. Common or Reusable Graphic Management

Graphics shall be managed, exported in external Cloud Drive and used across different software packages which are edge to plant SCADA to enterprise visualization.

The common graphics should provide context to data, enabling faster analysis and improved operator effectiveness. These graphics should be portable and cross platform compatible to support a variety of desktop clients, mobile clients, web servers, as well as cloud clients across a variety of operating systems. Moreover, these Graphics shall support storage and management in the cloud to drive enterprise standardization and re-use across teams and sites.

b. Graphics Editor

A graphics editor shall be included with the Development Environment and shall include a set of basic drawing tools to create simple or complex graphic objects. Selecting an icon on the drawing toolbar shall select a graphical shape primitive, and the editor will allow a user to then easily create simple graphical objects using these primitives. Primitives available shall include lines, rectangles, polygons, ellipses, circles, open curves, closed curves, two-point arcs and pies, three-point arcs and pies, and filled shapes or text.

Any of these objects shall be assigned various attributes such as line color, fill color, size, and orientation and shall be made static or dynamic. Text objects shall be scalable

and use true fonts in bold, italic or underline. All objects shall be scalable and moved in any direction one pixel at a time or dragged with a mouse. A standard palette of 48 pre-defined colors shall be provided. A user defined color palette shall be created, exported and imported. The color palette shall be based on 16.7 million colors. The system must also support the user choosing transparent colors for all graphical objects and backgrounds. Possible colors shall be solid, one, two- and three-color gradients, patterns, textures, and no fill. Gradient direction shall be vertical, horizontal, radial, point based and custom. Additional gradient controls shall control color distribution shape, peak and falloff, width, and height.

i. Graphics Manipulation

The graphics editor shall support standard object manipulation functions such as cut, copy, paste and delete. Alignment tools shall be included to simplify proper placement and arrangement of objects. Align commands shall be included to align objects based on justification to the left, right, center, top or bottom. Object commands shall also be included to space them vertically, horizontally, move to back, move to front, rotate or group and ungroup.

The graphics editor shall be able to modify a member of a group without breaking the group.

ii. Editor Pan and Zoom

The graphics editor shall be able to pan and zoom in the editing environment 75% to 500%.

iii. Vendor Supplied Graphics Library

The graphics editor shall include a broad library of complex objects and process symbols such as meters, pushbuttons, sliders, gauges, pumps, motors, tanks, valves, trends, alarms, controller faceplates and bitmaps.

iv. Reusable Configurable Library Symbol Editor

The graphics editor shall allow a user to develop his own library of reusable configurable complex objects and process symbols.

v. Graphics Scaling

All complex graphic objects shall be scalable to any size and may include animation links to provide dynamic response based on real time data or user action.

The graphics engine to support the development of vector-based graphics which do not pixelate as they expand and contract in size, maintain high resolution.

vi. Graphical Animation

The graphics editor shall support configuration of at least the following animations.

vii. Color Animation

The color of graphical symbol may be changed dynamically by Boolean, analog, or string references. There shall be no fixed limit on the number of color changes a single symbol may have. Percentage of fill for objects up, down, left or right direction based on a tag name or hierarchical Name.

viii. Blink Animation

A graphical object may blink based upon any Boolean expression, alarm, event, or upon a designated group of alarms. The blink shall be adjustable to slow, medium or fast.

ix. Alarm Border Animation

User shall have the ability to configure a border around a selected graphical element that can display the alarm status of a desired associated reference. The Alarm border shall blink when an alarm is active using a color corresponding to the severity of the alarm. It shall stop blinking upon a return to normal state and disappear once the alarm condition is gone. A badge icon associated with the corresponding severity of the alarm shall also be displayed during the animation.

x. Visibility and Transparency

Each object shall have a visibility and a transparency attribute option allowing for visibility/transparency of the object based upon the status of an analog or discrete point, alarm, or operator security level.

xi. Size, Location and Orientation

The system shall support animation of objects via re-sizing, moving, and/or rotating based upon a change in a tag name or hierarchical name.

Objects shall be animated based upon any user-defined criteria made up of tag names in the system. This includes the use of expressions containing mathematical functions and the status of analog and discrete values in the system.

xii. Layering

The graphics Editor shall allow layering of objects to activate specific objects based upon conditions in the process. The Graphics Editor shall be able to control

and visually see the Z-order of the layering of graphical elements.

xiii. Grid

Graphics development tools shall allow object placement via a "snap-to-grid" feature with configurable grid spacing.

xiv. Graphical Elements Naming

All graphical elements shall be named. The user shall be able to change the system supplied name. The name shall be able to be referenced in animation logic.

xv. Editor Undo/Redo

Graphics development tools shall support an "undo/redo" feature with a configurable number of levels and command displays.

xvi. Graphical Element Styles

The development environment shall allow central configuration of a standard set of graphical element styles so as to help consistency in graphical design throughout an application, including aspects such as text font, color and size; line color, weight and thicknesses, shape fill pattern or color, outline pattern and color as well as blink conditions.

xvii. Numeric Format Styles

The development environment shall allow central configuration of a standard set of numeric format element styles so as to help consistency in display of numbers throughout the application.

xviii. Intelligent Graphics

The system shall provide a library of "self-configuring" objects that change properties based on dialog box entries made during development. For example, consider a standard setpoint loader object that has a graduated scale and a default range of 0 to 100.0. Dialog box entries shall allow changes to the range of the setpoint loader, the number of major and minor divisions on the scale and change the text font used for the labels. The object shall then redraw itself with new number of tick marks, new spacing, new labels, and new font.

Object Wizard is an interface that shall be added to a template (Asset Standard) to assist the user in configuring derived instances (Assets). The Object Wizard contains prompts or questions to guide the user through the steps needed to configure the Asset.

Features of Object Wizard includes:

- Simplified workflow for configuring instances
- Changes propagate automatically (template locking is not required)

- Unnecessary attributes shall be trimmed from instances to reduce the footprint of deployed objects
- Unused symbols shall be hidden from users to reduce clutter
- Reduced system maintenance
- Reduced training requirements for plant personnel
- A template with an Object Wizard functions as a self-configuring template. The Object Wizard contains choices and options that guide users through the process of configuring instances.

xix. Drawings and Images File format.

The graphics editor supports importing .bmp, .gif, .jpg, .jpeg, .tif, .tiff, .png, .ico, .emf. Format support for .pcx and .tga are not natively included as they are older formats, but BOTH are easily converted to .jpg, which is supported.

xx. Copy/Paste

The graphics development environment shall support the copy of single or multiple animated graphic objects and symbols with just two keystrokes, and immediate substitution of tag names for the duplicated object shall be possible without leaving the graphics editor. The graphics development environment shall support the copy of single or multiple animated graphic objects and symbols from one window or display to another retaining all the animation characteristics, links and attributes. This function is to eliminate duplication of effort. In addition, it shall be possible to import windows from another application in this same fashion. The graphic editor shall support the copy and paste of animation configuration from one graphical element to another. User shall have the capability to add tag name or hierarchical name entries while building a display without exiting the graphics editor.

xxi. Help Subsystem

Context sensitive help shall support display building. Users shall be able to obtain immediate help on all configuration subjects by pressing a single function key.

There are actually two contextual help options to leverage. "F1" for help on the specific area, such as the graphic editor, OR "Ctrl F1" for specific help on a selected object or graphic.

xxii. Graphics Design

The user shall be able to define graphic screens while the system is monitoring the process.

xxiii. Graphical Application Deployment

The user shall be able to distribute the visualization application and/or changes to single or multiple nodes. View Engine Objects are deployed to any node requiring HMI functionality. Moving applications from node to node is as simple as un-deploying, dragging and dropping to another View Engine, then redeploying.

c. Graphical Applets

Application developers shall be able to create resolution independent self-contained graphical “applets” with integrated scripting, their own namespace and custom attributes. The scripting language used shall be the same as that for the main HMI logic scripting.

It shall be possible to use system calls from the .NET library within the scripting language. The graphical applet scripting shall consist of an On Show, On Hide, While Showing (periodic), and user defined and named On True, On False, While True (periodic), While False, and Data Change scripts. The Application Developer shall be able to define local and external named attributes of the following data types: Integer, String, Boolean, Float, Double, Time, and Elapsed Time.

d. Graphical Applet Manager

Using a Graphical Applet Manager, application developers shall be able to create groups of graphical symbols that then comprise a single named Symbol that may be expressed as a named Template. There shall be no practical limit to the number of graphic symbols that can comprise a single Compound Symbol, up to, and including an entire window of symbols. These templates shall connect either to Object Database objects, to local HMI tags or to HMI tags through remote references. Changes to the Graphical Template shall propagate to all instances of the template. Libraries of re-usable Graphical Applets may be created, organized and managed using a Symbol Manager.

e. Graphics Organization

The Graphical Applet Manager shall allow for the creation and deletion of a hierarchical tree of folders. Graphical Symbols may be moved between folders via drag-and-drop.

f. Graphics Import and Export

The Graphical Applet Manager shall allow for the import and export of Applet Symbols.

g. Graphics Management

The Graphical Applet Manager shall allow for the renaming of Graphical Symbols. Symbols can contain symbols that contain symbols. There shall be no fixed constraint on the levels of containment. A change to the parent of an embedded symbol shall update all instances of the contained symbol.

A Graphical Applet Symbol shall be instantiated in the HMI window by selecting the symbol in the Symbol Manager and selecting OK. The selected Symbol shall invoke a dialog to allow for browsing and selecting new tags or attributes from the Object or Tag Database, and to allow for creating entire new named objects in the object database. The Symbol may then be resized by dragging a resize handle the symbol to the desired size.

The Graphical Applet Manager shall allow for the editing of a Symbol. At the completion of the editing process a dialog shall be displayed that allows the changes to be committed, aborted, or return to editing. The tags/objects attributes of a Graphical Applet Symbol may be changed or redirected to an alternate set of tags/object attributes at runtime via a single script command.

8.7.3 Navigation Tree

This software shall allow user to select the built-in navigation app which is based on hierarchical model creates and update. This app allows standard HMI to navigate on selected object in this navigation tree app, ability to hide objects based on certain character like `_' etc.

Role-based Navigation Model: The application navigation model should adapt to reshape based on the logged-on user's assigned roles.

8.7.4 Alarm Display

Alarms shall be displayed by configuring a user-defined alarm summary object, which may be placed by itself or along with other objects in a window. The object shall be sized and then double-clicked to launch a configuration dialog. Default alarm object configurations shall be displayed with the option of changing any configuration parameter for runtime viewing.

The alarm object configuration shall include parameters with check boxes to select and enable or disable how the alarms appear at runtime. Alarms shall be color coded according to the state and priority of the alarm including an acknowledged alarm, unacknowledged alarm, and an alarm that has returned to normal but is not yet acknowledged. The user shall be able to choose from 256 different colors for display of each of these alarm states. The alarm display object shall also support event display, with the color used for events also being one of the 256 different colors.

Alarms shall be able to be shown either as real-time alarms or as historical alarms. The same object shall communicate with the live process or the alarm history database. Alarm data shall be able to filter by updating custom query using script or during runtime and contextually aware.

8.7.5 Alarm/Event and Trend overlay

The trend client should allow to overlay alarm and event information and enable ease of troubleshooting.

8.7.6 .Net/WPF/WebControl support

System should support .Net, Web, ActiveX and WPF Control technology, the software shall provide extensibility by providing integrated support for these controls. The system shall automatically distribute these assemblies and dependencies to client nodes during the publish / deployment process.

The SCADA vendor shall provide a solution that facilitates the use of 3rd party controls to enhance the application. The application shall provide a container for ActiveX, .NET (WinForms and WPF) as well as web content (HTML5 and JavaScript)

8.8 Advanced Context Driven Apps

The system should be open and flexible to build new Apps via open SDK or utilized existing apps like documents, GIS, Camera, AI App s which are context driven. These Apps let user utilized the apps which shall be driven by the context of your overall screen to display the right content. Some of the Apps are listed below:

8.8.1 Document Viewer

The SCADA vendor shall provide a native document viewer application that enables users to view documents based on selected context of the asset.

8.8.2 Geospatial Mapping

The Software Solution shall offer geographical contextual presentation which supports model-based navigation. The Map Application should support secure GIS overlays and include support for major map servers such as Google Maps, Bing, Open Street, Baidu and ESRI. The Map application should support boundary configuration, as well.

8.8.3 3D Viewer

The software should render 3D models of assets contextually in HMI. The 3D models should allow users quick and intuitive understanding on asset context of alarms, alerts, and status changes. Users should be allowed to visualize the impact of operational information in the context of the physical surrounding of the asset.

8.8.4 Content Presenter

The Software Solutions shall provide the user an ability to dynamically create runtime dashboards with KPIs, symbols or alarms status of areas.

8.8.5 Playback

The system should have a capability to select any process screens and playback on selected interval and speed. This should help users to troubleshoot the specific issue.

8.8.6 Anomaly Detection

The Software Solution should support building artificial intelligence into the context of real time decision making and provide anomaly detection.

8.8.7 Application to view PLC Code

The software can potentially have integrated Application to view PLC Code that allow engineers to trouble shoot PLC lock-ups in real-time.

8.8.8 Video Integration

The Software Solution should provide for ease of video (camera, CCTV) integration with context based on asset or selected object. This function should show video and share any alarm or event information.

8.8.9 HMI Application Management

The SCADA system Development Environment shall be able to manage the visualization applications, as well as capabilities to distribute the selected HMI application(s) by deployment via drag and drop.

Changes to a visualization application shall be propagated to any instances through the entire SCADA system. The SCADA system shall also include an application manager with a Windows Explorer-like browser to simplify management of client applications. The Application Manager shall provide the capability to dynamically change the resolution of the application windows. This will allow graphic displays to be developed on workstations with different display resolutions and convert them to the desired resolution quickly so that they are all consistent in look and feel.

8.8.10 Distributed Network Application Management

The SCADA software shall provide standard functionality that will simplify the configuration, operation, troubleshooting and maintenance of the application by providing means of easily distributing the application in network environments. The management software shall allow a single master application to be developed and maintained on the network. The Development Environment shall allow automatic distribution of the master application to all nodes on the SCADA control network as well as propagation of changes to the master application to all its instances in the system.

When a client node is hosting a deployed HMI application, the client shall maintain a copy of the application on its local hard drive and become registered as a user of that application. When a change to the template application is detected, each registered user node shall be notified of the change.

The Design Environment shall allow the user to define how the client node is notified of the change in the application.

The client node shall either automatically load the new application, prompt the user to load changes or ignore, or automatically ignore such changes. If a network failure occurs between the repository and client, then the client shall continue to run the last distributed application. When the network is restored and the server application has changed, the system will distribute the server application to the client.

8.8.11 Multiple Instances

The system should allow to execute multiple instances of View App applications on one node or server.

8.8.12 Multi-Monitor Support

The system should have multi-monitor support for visualization with no scripting.

8.8.13 Troubleshooting Application

The software shall include capability to continuously monitors deployed applications and system hardware, identifies upset conditions and alerts potential issues before they manifest into real problems like software application errors or server / client machine downtime events.

8.8.14 Software Asset Management

The software shall include software asset management capabilities to help users to manage the software and its licensing at each site. It should discover installed software and licenses and creates detailed inventory reports.

8.8.15 Application Log Files

Application log files for diagnostics and troubleshooting purposes shall reside on the local hard drive for a user-defined number of days. Each network node shall maintain an independent log file for the applications that are unique to each node. A new log file will be created and archived daily according to the user specified time and location.

A viewer snap-in to the Microsoft System Console utility shall be available for viewing Event logs. The viewer should support color distinctions for different threads, processes, or programs. The log file viewer shall support viewing local and remote node application log files.

8.9 Additional HMI capabilities

8.9.1 Hardening web security

- Web client Reverse Proxy Support for Secure Web Access
- Use any 3rd party reverse proxy to access Web client from outside the control network.
- Web client integration with Identity Manager
- Web client is integrated with Identity Manager. You can manage user authentication from a central location.

8.9.2 Performance & User Usability

- Web client Per-User Memory Tags
- Configure any HMI Memory tag to have a value per user in Web client. This ensures the usability of the web client to support multiple users to view the same HMI application concurrently.
- Web Client Performance
- Web graphics rendering time and resource utilization should not be significantly different from desktop client
- Anonymous Connections with IFrame Connections
- Simplifies access to 3rd party portals and websites embedded in graphics removing the need for entering user credentials for specified graphics. This ensures usability when using read-only access to graphics in 3rd party portals and websites.
- SQL Data Grid Support in Web Client
- A SQL Data Grid control in an HMI application is transparently replaced by a web widget based equivalent using the same configuration for display to render in the Web client. Seamless without any re-engineering.

8.9.3 Empowering Connected Workers Beyond HMI

- OPC UA Server
- Provide real-time process tags data to enterprise and business systems using industry standards.
- Mobile App Support for Different Web Servers
- A mobile application shall be available that allows users to connect to any of the visualization web servers and present the user interface on both iOS and Android based mobile devices.
- QR Code Scanner web widget
- Use a mobile or tablet camera to scan a QR Code from HMI and web client to dynamically bring up an HMI display or faceplate, operator work instruction or any other contextual information.
- Web Browser web widget
- Google Chromium based browser allows you to access modern websites from HMI and web client.

9.0 The Runtime Environment

This section describes the various user interface functions of the system in the runtime mode.

9.1 Alarm Management

Alarms shall be detected and reported by an Alarm Manager Service. The Alarm Manager Service shall support no less than two hundred (200) simultaneous alarm client displays. In the event of an alarm storm (hundreds or thousands of alarms detected within one second), the Alarm Manager shall report, and the client shall be capable of displaying up to one thousand (1000) new alarms within ten (10) seconds of the detection of the alarms.

System shall provide the ability of shelving alarms in order to enable authorized operators to temporarily remove selected alarms from the list of active alarms suppressing them for a specified period. System shall request operators to enter a reason for shelving. System shall provide the ability to optionally suppress alarms based on selected plant states in order to avoid displaying invalid nuisance alarms during specific states.

The system shall have the ability to provide the total number of aggregated alarms of each severity category (critical, high, medium and low) under each area level. The system shall be able to alarm system resources (CPU utilization, memory, etc.).

Alarms shall be able to be logged to a Microsoft SQL Server database. Alarm events to be recorded shall include alarm instantiation, alarm return-to-normal, and alarm acknowledgment.

Items to be logged in addition to the alarm event shall include date and time of alarm event, Alarm Group, Alarm Tag name, Alarm Tag Type (real/integer/Boolean), Alarm Type (LoLo, Lo, Hi, HiHi, ROC, Deviation, etc.), Operator Name, Operator Node of alarm acknowledgement, and Alarm Priority.

An Alarm Purge service shall be provided to automatically purge and optionally archive alarms that are older than a user defined period of days online.

Alarms may be printed to a locally connected or network printer. The alarms printed from a particular node may be all alarms, only unacknowledged alarms, only acknowledged alarms, alarms from a particular alarm group or groups, alarms from a particular priority to a particular priority or alarms from multiple alarm providers.

9.2 Communications Architecture

The runtime environment shall be based on distributed, peer-to-peer system architecture. It shall be possible to scale the architecture from a single, self-contained node, to over 100 local nodes. The architecture shall contain a multi-computer model that is seen as a single distributed namespace in the runtime environment and does not require replication of data from one node to another.

Application Objects and their attributes shall be accessible by the objects Hierarchical Names, or globally unique tag names.

9.2.1 Communication Drivers

The SCADA system shall include a broad range of Device Integration servers for establishing the I/O interface to field devices such as RTU's, PLC's and DCS systems or IIOT Devices. The software system shall have the ability to connect to any I/O server utilizing the SuiteLink or OPC, OPC UA, MQTT- SparkPlug B, BacNet, Profibus, Modbus protocols. I/O shall be defined as any input and/or output variable, including individual data acquisition points and any variable parameter generated for exchange between objects in the system. At a minimum, the data types that shall be supported are Boolean, Float, Double, String, Internationalized String, Integer (8, 16, and 32 bit, signed and unsigned), Time, and Elapsed time. The software shall enable developers to expose runtime data to OPC-UA Clients directly (no gateway required) including browsing and OPC-DA.

a. General Purpose Device Integration Servers

General-purpose Device Integration servers shall be available for all major PLC's from Allen Bradley, GE, Modicon, or Siemens, plus various RTU's and DCS systems. The PLC communication servers shall support interfaces via direct serial, local control networks such as Data Highway Plus, Modbus Plus or via TCP/IP Ethernet. There shall be support for several hundred various devices utilizing the protocols of SuiteLink, OPC (DA), and OPC UA (DA). A Data Access Server toolkit shall be available for third parties to develop custom I/O servers.

b. Supervisory Drivers (General Purpose Drivers)

- General-purpose Communication Drivers servers shall be available for all major PLC brands, such as Schneider-Electric, Allen-Bradley, Siemens, and GE
- Additional support for Automation Direct, Bosch, Eaton, WAGO, Beckhoff, BACnet, Texas Instruments, Mitsubishi, Omron and Opto22.
- The PLC communication servers shall support interfaces via direct serial, local control networks such as Data Highway Plus, Modbus Plus or via TCP/IP Ethernet.
- System should allow to communicate with other devices utilizing the protocols of SuiteLink, OPC (DA), and OPC UA(DA).

- Communication Driver should have Built-in redundant network path communication to devices
- The Drivers are directly installable as optional selections from the installation media
-

c. IIoT and cloud-ready Communication Drivers

- IIoT and cloud-ready Communication drivers shall be available for communicate with web- based applications and IoT devices using protocols like OPC-UA, MQTT and SOAP/REST/oData web services.
- Support for MQTT standard specification including support for
 - Simple and complex JSON payloads
 - Windows/Linux timestamps
 - File transfer inclusion in payloads
 - Serialized Date/Time File transfer in payloads
 - Store/Forward upon network disconnects and reconnects when publishing data to the broker
- Additional support for MQTT Sparkplug Specification including, but not limited to:
 - Topic Namespace Standardization ▪ TCP/IP
 - Device State Management (Birth, Death, Last Will Testament, Persistence, Store/Forward, Multiple Broker/Server Support)
 - Payload standardization (following Google protocol buffers encoding scheme)
 - Security (Unique Client ID, TLS, User Authentication and Authorization)

d. Telemetry Communication Drivers

- Telemetry Communication drivers shall be available for communicate with Intermittent Comms Links using native built-in drivers such as DNP3 (Level 2 & 3), IEC 60870 and Modbus (TCP, RTU, ASCII). Include both master and slave support for the above-mentioned protocols.
- Shall be able to leverage the following communication media support
 - TCP/IP
 - UDP/IP
 - Ethernet Via Radio
 - Ethernet to Serial
 - Serial (RS232 / RS485)
 - Dial-Up (Modem)
- The communication functionality shall include
 - Independent Ethernet Communications
 - Round-Robin Polling ethernet Communications

- On-Demand Ethernet (Scheduled)
 - Configurable Interval Polling
 - Configurable Polling Offsets
 - Configurable Escalation
 - Automatic Dial-Up Configuration
 - On-Demand Dial-Up (Scheduled) Configuration
 - Multiple Schedule Management
- Data Capture
 - Polling Engineer
 - Low Bandwidth Optimization
 - DNP3 Buffered Events and Data Support
 - DNP3 Time Synchronization with RTU
 - Unsolicited messaging support
- Fault Tolerance
 - Dual redundant communication paths
 - PSTN fallback
- Telemetry Server dual and Triple Redundancy
- Application Management
 - Configuration Management of Connectivity Protocols
 - Flexible Organization (Configurable Grouping)
 - Templates Support
 - Manage common configurations
 - Propagate changes to instances
 - Lock parameters to templates
- Project Backups
 - Export/Import of application configuration
- Security
 - Secured by default
 - Built-in Customizable Application Security
 - Configurable Users and Groups
 - OS security integration
 - Active Directory and LDAP Integration
 - Customizable permissions

e. Communications Extensibility

The vendor shall permit third party communication software vendors to provide Device Integration capability to the SCADA system. Many of the Device Integration Drivers come from 3rd Party Vendors, such as Software Toolbox, Matrikon, ConnexSoft and Kepware with no limitations in use.

f. Toolkit/SDK

A toolkit shall be available for third parties to develop custom Communication Drivers.

g. Multi-Protocol Communications Gateway Server

A component shall exist to acts as a communications protocol converter and shall be used to link clients and data sources that communicate using different protocols like SuiteLink, OPC DA, OPC UA and MQTT.

9.3 Runtime Data Viewer

The system shall provide a utility to view the real-time status, quality and value of any Application Object Attribute.

9.3.1 Email Support

The SCADA solution shall provide the ability to send email, with attachments, or text messages that can be accessed from mobile devices, enabling the user to receive real-time information on alarms, process values and events. Full runtime shall support SSL encryption.

9.3.2 Events

The SCADA solution shall ensure traceability for operator-initiated actions, or internal system activities. The system shall log events such as security system changes, screen open/close, recipe/report operations or system warnings.

9.3.3 FTP Support

The SCADA solution shall automatically upload and download files during runtime to/from remote storage locations using the FTP protocol.

9.3.4 IoT View Support

The SCADA solution shall provide a platform agnostic core runtime to support Linux, VxWorks and other embedded platforms.

9.3.5 Multi-touch Interface

The Runtime environment shall support familiar, modern interface gestures, like pinch zooming and panning, swiping, rotate and dual-touch commands.

9.4 SCADA System Failover

The SCADA system software shall provide high availability for all functions within a normal SCADA controls environment. The specific components that require redundancy within the SCADA system are Application Object and Application Object Hosting, PLC/RTU communications and alarm reporting. High availability requirements

also apply to logging of historical process data. In redundant failover configuration, there shall be a Primary and a Backup system object that manages contained Primary and Backup objects. The system shall execute active objects and synchronize active objects with standby objects. In the event of detection of any failure in active object execution or communication with the active object, standby objects shall begin executing and communicating within the system.

9.4.1 Defined Failure Events

The SCADA system shall detect the following events within the SCADA system and network objects:

- Communications failure to a single RTU/PLC
- Communications failure to multiple RTU/PLC(s)
- Communications failure to Communications Server
- Application Logic Failure
- Alarm Printer Failure (Off-line, out of paper) Alarm Manager Failure
- Communications failure to data historian
- Data historian rate of collection deviation
- Low Disk Space on any data historian on the network

The SCADA System shall detect any or all the possible failures and allow client data recovery without operator intervention.

9.4.2 Application Redundancy

The system shall provide for the execution of standby application objects that become active upon the failure of execution of active objects or failure to communicate with the active objects.

Separate configuration of standby objects shall not be required. Objects shall be allocated to a Primary object manager engine, which in turn ensures that contained Backup objects are created and deployed as standby. In normal operation, the Primary engine along with its contained objects shall be active. The backup engine and contained objects shall be kept in standby and shall be synchronized with their active partners on a user configurable time base. Configuration of application redundancy shall be by configuration by check-box and drag-and-drop. Application redundancy shall be a built-in out-of-the-box capability. Custom scripting to achieve this basic functionality is not acceptable.

A failover is the condition during which run-time operations are moved from one critical component to another. Failover can occur due to failure conditions or it can be forced manually, called a forced failover.

9.4.3 Alarm Redundancy

The system shall provide for the handling of alarms from standby application objects that become active upon the failure of execution of active objects or failure to communicate with the active objects. Separate configuration of alarms in standby objects shall not be required. As with application redundancy, objects shall be allocated to a Primary manager object that in turn ensures that contained standby objects are created and deployed as standby for handling of alarms.

A dedicated Ethernet connection shall be required between the platforms hosting redundant applications. This connection shall support the need to keep both the active and backup application synchronized with alarms, history, and checkpoint items from the application that is in the Active Role. Each application also uses this Message Channel to provide its health and status information to the other.

9.4.4 Communications Redundancy

The SCADA System shall monitor the status of communications to the Communications Server(s) and the status of the Communications Server to each PLC/RTU. In the event of a communications failure, the SCADA System shall transfer communications responsibility to a designated standby communications server.

9.4.5 Data Historization Redundancy

The system shall provide for historization of data values from active objects. Upon the failure of execution of active primary objects, standby objects shall be activated and assumed the task of providing data for historization. Separate configuration of historization for standby objects shall not be required. If the historian is off-line or unreachable, the engines servicing active objects shall store the historized data locally and forward the buffered data to the historian when the historian server is available. Primary and standby object engines shall synchronize any buffered historization data. If the historian is off-line or unreachable and the primary object engine fails, the failover engine shall assume that task of storing the historized data locally and forward the buffered data to the historian when the historian server is available. There shall be no practical limit, other than disk space, as to the size of the historized data stored locally.

The Data historian shall be a store-and-forward method to protect against data loss if communication is interrupted between the data source and historian. If the data source loses communication with the Data historian, the source

stores the collected data until communication is re-established. Then, it forwards the stored data to historian.

9.4.6 Remote Desktop Server (Terminal Services) Fail-Over

Remote Desktop thin clients shall be capable of automatically failing over to a redundant terminal server. No operator intervention shall be required. The system shall support execution of the visualization software and the engineering development tools in Terminal Services sessions while enforcing the configured operating system security model.

9.4.7 Browser support

For Remote Desktop systems, an option shall be available to additionally access the visualization application by exercising concurrent remote desktop sessions via a web browser with HTML5 support.

9.5 Runtime Security

Security of access is paramount in the Runtime environment. The runtime environment shall be able to be configured such that operators of different abilities, roles and responsibilities are permitted or denied access to system functionality as appropriate. The security constraints shall be provided by inherent capabilities of the SCADA software and not by the engineered application.

9.5.1 Changes to runtime data

Runtime changes to object values shall be subject to security authorization. Permissions that are configured using the Development Environment shall be automatically checked at runtime for authorization including verification of identity and access permission related to the originator of the runtime change request. Failed authentication requests shall be logged.

The runtime architecture shall conform to the object attribute security model defined in the configuration environment.

9.5.2 Login Requirements

Users shall log in before any change to any object attribute that has been constrained is allowed.

9.5.3 Runtime Audit Trail

It shall be possible to configure the system such that any runtime changes to a variable shall provide an audit trail of user ID, full user name, previous value, and new value.

Attributes configured for Verification shall provide an audit trail of user ID, full user name, verifier username and full user name, previous value, and new value.

9.5.4 Operator Security

The Operator Workstation shall utilize the security model defined by the configuration database.

The software shall utilize data level security where the ability to modify a setpoint or other value is determined in the configuration database. Any changes to the data level security model shall be seen by all operator stations without any modifications to the operator stations.

The security system shall be capable of disabling access to all Microsoft Windows controls (file menu, close, minimize, etc.) and keyboard commands (Ctrl-ESC, Alt-Tab, and limiting external system access via Ctrl-Alt-Del).

a. Logging Operator Actions

All operator actions shall be able to be logged to an event logger. The event logger shall keep track of each new operator log-on, log-off, setpoint change, or device control.

Each event log shall record the date, time, operator logged in and the type of action taken (setpoint change, state change, etc.).

b. Credential Management

The software shall be able to provide a method to retrieve list of named credentials (not the actual credentials) available to the current user context in the App or Layout Editor for credential management.

9.5.5 Value Change Event Logging

Any configured Integer, Real, Discrete, or String tag may also be configured as the source for an event to be triggered. The point shall be logged as an event any time its value changes. Events shall be logged to a Microsoft SQL Server database. Items to be logged in addition to the event itself shall include date and time of the event, and Event Priority.

9.5.6 Intellectual Property Protection

The SCADA solution shall provide the ability configure the system to protect intellectual property such as screens, documents, scripts and worksheets by individually password protection.

10.0 Full Function Operator Workstation (HMI)

The SCADA system operator shall be able to execute all monitoring and supervisory control functions from this workstation. Typical operator commands include modifying setpoints for control loops, alarm acknowledgment and setpoint adjustment, auto/manual switching and on/off control of field devices and taking points or devices on/off scan.

The operator shall be able to access all SCADA tag name or hierarchical names or graphic displays from any workstation on the network without having to know which data historian or server the point or display resides on.

10.1 Workstation Hardware

The SCADA software shall be capable of running on commercially available computer systems running a supported operating system. The system should be capable to do silent install.

The SCADA software vendor shall have a program for certifying third party operator interface panel hardware and it shall be possible to purchase the vendors' software pre-installed on such hardware.

The SCADA software vendor shall offer as a standard product on their standard price list a selection of suitable HMI panel hardware available with the SCADA software preconfigured.

The panel hardware shall be able to run Windows Client operating systems, including Windows 7 Embedded and Windows 10 Core.

10.2 Thin Client Operator Workstation

The SCADA system operator shall be able to execute all monitoring and supervisory control functions from a thin client workstation or environment. No SCADA HMI software shall be required to be installed on a Thin Client Operator Workstation. This workstation shall require only the firmware or software required to initiate a Remote Desktop session. The SCADA HMI shall support Microsoft Windows 2012 R2 or later Server operating systems Application Server Mode utilizing native Remote Desktop Protocol (RDP).

The SCADA HMI shall support up to seventy-five (75) sessions of the HMI software running on a single Terminal Server. No modifications to the SCADA HMI configuration shall be required to allow running in a thin client configuration. The exact same application running on a Full Function Operator Workstation (thick client), shall run in a terminal services (Remote Desktop) session.

A management interface to the thin client terminal population shall be provided. It shall be capable of remote monitoring, reboot, failover and screen shadowing. The thin client hardware shall be capable of supporting multi monitor operation.

10.3 Process Data Analysis Workstation

The SCADA system software shall include a set of easy-to-use client software tools for real-time and historical data analysis and system reports. This client analysis software may be used by engineering, maintenance or supervisory personnel who need information from the SCADA system but do not require access to graphic displays. The client tools shall be able to access data from multiple data historians on the SCADA system network.

Users shall be required to log in with a password to access the database server. The user shall not have to know the location of the server on the network, only the name of the server is enough. The data analysis software shall include tools for advanced trending analysis, X-Y plotting of tag names, and viewing of reports in spreadsheet or free form format. All tools shall support full right mouse- click capability for quick menu selection of available functions. The client tools shall be available as a stand-alone program or as a control (ActiveX, .NET WinForms, WPF or Web) for embedding into the SCADA displays so that any full function or view only operator workstation may have the same capability, or any user interface that supports the control.

10.4 View-only Graphics Display Workstation

The SCADA system software shall support view-only graphics workstations for managers or supervisory personnel who wish to have access to all displays and trends but do not have process control or alarm acknowledgement responsibilities. View-only applications are essentially 'read- only'.

No modifications to the SCADA HMI application or configuration shall be required for this functionality. This functionality shall be enabled by software license and shall not be able to be overridden. The view only graphics display HMI shall be capable of running in a terminal services session.

View-only graphics are default when no user has logged on to the system. Upon logon, access is controlled based on username or user-role privileges. This is highly configurable, with no scripting, and applicable down to single variables.

11.0 Data historian

The SCADA system software shall provide a real-time relational database historian for long-term storage of process data. The Data historian shall provide for the storage of real-time and historical data for each analog, discrete or string tag name. The data historian shall also store summary, event, alarm, and configuration data.

The data historian database shall acquire and store process data at full resolution, with options to store on time dead-band, value dead-band or Swinging Door. The data from historian database shall include normalized extension tables for real time data and include a set of client tools for data analysis and reporting such as those described in earlier sections.

The Data historian shall be capable of running in a stand-alone mode without connection to or configuration from the SCADA system. While there are always physical limiting factors such as disk space, there shall be no programmatic limit to the amount of data that may be stored on-line. Additionally, there shall be no performance penalty for long-term data storage. There shall be no discernable difference in retrieval speed of data based on the age of the data. For example, the retrieval of two hours data stored two years prior shall be the same as for two hours of data stored one day ago.

The system shall have the option to also store Alarms and Events history in the same historian as the process data.

11.1 Historian Architecture

The historian shall be able to be configured to meet the needs of data acquisition and consolidation over a wide area network.

11.1.1 Remote Station Acquisition and Analysis

A historian shall be able to be configured at a remote site to acquire data locally, and to be able to transmit that data to a second historian in a tiered network architecture. Options shall be available to replicate all data, a selected subset of data or summarized data to a tiered historian.

Data shall be able to be stored locally at the remote site and analyzed with the vendors' client tools. Data shall be able to be aggregated and transmitted to a second level historian for further analysis.

Multiple remote historians shall be able to send aggregated or raw information to a single second tier historian or shall be able to send the same information to multiple second tier systems. Multiple tiered of historian are also configured to collect data from edge or site to region to HQ and corporate. The historian shall be designed to tolerate and behave predictably and consistently within a networked architecture if the network communication is slow or intermittently interrupted. A historian shall be able to accept data delivered from a remote, disconnected, location once

connection is restored. Data shall be able to be stored on a remote collection system and forwarded to the historian without operator intervention on restoration of network connectivity. It shall be possible to configure the historian architecture to guarantee no information loss due to network connection failure.

Data from one historian shall be replicated to one or more historians, creating a "tiered" relationship between the historians. Historian can replicate process data as well as alarms and events. A historian can act as a tier-1 and a tier-2 historian simultaneously.

11.2 Data Compression

The database shall support high-speed data acquisition and efficient data compression. The data compression for the data historian shall not use any algorithms that do not allow for the storage of the tag data at their scanned rate. The stored data records shall be able to recreate the process data in a loss-less format. Below are the data storage techniques to be employed for the various types of data.

- Discrete Data storage.
- Analog Data storage.
- String Data storage.

The historian should combine the power and flexibility of Microsoft SQL Server with high-speed acquisition and efficient data compression to store time-series data.

11.2.1 Discrete Storage Requirements

Each discrete stored value written to the Data historian shall use approximately 8 bytes of space. Raw data is compressed and stored within the Windows Explorer file structure.

11.2.2 Analog Storage Requirements

Each delta stored analog value written to the Data historian shall use approximately 10 bytes of space. Raw data is compressed and stored within the Windows Explorer File structure.

11.2.3 String Storage Requirements

Data shall be at a minimum capable of storing string, or text, data. Each string may be up to 512 characters in length.

11.3 Data Quality

With each value stored in history, a data quality indicator shall be stored as well.

11.4 Standardized Database Tables

The process of setting up the database tables shall be automatically configured and require no database engineering. Data definitions including the creation of database objects, such as tables, indexes, constraints, defaults, rules, stored procedures, triggers, and views shall follow a standard, published and readily available database schema.

This standard database schema shall outline the relationship between tables, table columns, keys and indexes and shall allow for third party development of client applications. Database device sizes shall be dynamically allocated during database installation depending on the number of tag names to be stored within the data historian.

All necessary database tables, relationships and stored procedures are created during installation, and are normalized to ensure peak performance.

11.4.1 Database Summary System

The data historian shall provide user configurable data summary for analog, discrete, and string, and to automate the collection of aggregate historical information based on a declared event. Pre-defined summary schedules of one, five, fifteen minute, hourly, shift, and daily shall be provided. Custom schedules may be defined by the user.

Analog Summary shall contain beginning value and timestamp, maximum value and timestamp at max, minimum value and timestamp at min, average, standard deviation, integral, and percent of good values in the summary calculation.

Discrete, Integer, and String State summary shall contain the total transitions into a state, total, max, min and average times in each state. Integer and String shall support a minimum of ten states.

In addition to the ability to access summary data via SQL, the system shall provide the capability to easily access summary data for HMI visualization purposes.

11.5 Single Configuration of Data Historization

The historization of data from SCADA objects shall be entered once at the time of configuration of those objects using the corresponding object editor in the SCADA System. The Data historian shall automatically acquire those configuration historization parameters upon deployment of the configured Application Objects.

Configuration shall provide the option to work with a dual historian configuration in order to send the same data to both historians if desired.

11.5.1 Historical Data Store and Forward

If the historian is off-line or unreachable, the engines servicing active objects shall store data locally, and forward the buffered data to the historian complete with time stamps and quality information when the historian server is available.

11.5.2 Shadow Copy (Live Backup support)

Support for Microsoft Windows Volume Shadow Copy Service (VSS), enabling consistent backups without shutting down the historian.

11.5.3 Tags, Expressions and Calculations

Historian can capture very high-resolution data on a large number of data tags to solve the most difficult process and equipment problems. Create user defined calculations and expressions with a pre-defined library of functions to create trending, graphical display and reports.

11.5.4 AI Tag Inference

- Forecast future state of assets and sensor values.
- Determine it can be made to the next planned maintenance outage.
- Provide key input for risk assessment.

11.5.5 Batch Reports

Operators, engineers, and decision makers can get an instant snapshot of what is happening during any industrial process. They can view and analyze reports and data trends on their desktop, laptop, or web browser through applications such as Microsoft Office, HMI , or even custom software clients.

11.5.6 Alarm Pareto Chart

Using the Alarm Pareto control, user can analyze which alarms and events occur most frequently in a given production system. Also analyze alarm frequency by the time periods during which they occur. The analysis capabilities of the Alarm Pareto control identify the largest issues of production systems. The Alarm Pareto control helps to recognize where to keep more focus to achieve the most significant improvements. The Alarm Pareto control shows a bar chart representing alarm activity.

11.5.7 Alarm History

Expanding on the alarming integration features, integration of alarm history with historical overlays directly from HMI with simple configuration, as an alternative to alarm database logger.

11.5.8 OPC UA Client Tag Prefix

Allows adding a prefix syntax to an alias set to simplify referencing of addresses that would otherwise be lengthy due to OPC UA syntax.

11.5.9 5.5.10. Backfilling Data

There is often a need to backfill old data from a tier-1 historian when a tier-2 historian is added. This utility shall be used to replicate data to an online or on-premises tier-2 historian. It is used to backfill old historian data to Insight.

11.6 Historical Data Point Configuration

The data historian shall include a database editor to modify the parameters of any tag name without using the SCADA database editor as an option. It shall be possible to configure the data storage rate for each point based on a user-defined rate frequency (cyclic storage) or upon change (delta storage). Cyclic storage rates shall be configurable per point from 1 second up to hours. The historian database shall support a one millisecond resolution for tag names configured with delta storage.

11.7 Historian Data Acquisition and Retrieval

The data historian shall acquire data via automatic and manual methods. Automatic data acquisition shall be through industry-standard data transports. Data Acquisition OLE for Process Control (OPC), in addition to proprietary transports, shall be supported. The method for retrieving data shall be Structured Query Language (SQL). It shall be possible to store data at one resolution and query at another. Methods shall exist to query and retrieve data cyclically, with millisecond resolution, no matter the storage mode. It shall be possible to query and retrieve data in delta, with user selected dead-band criteria and millisecond resolution, no matter the storage mode. It shall be possible to query and retrieve evenly spaced data over long periods of time where the criteria are a row count, no matter the storage mode. It shall be possible to query and retrieve all stored values. It shall be possible to query and retrieve analog and state summary values, best fit, slope, and integral summary values.

11.7.1 Dynamic Configuration and Bump-less Data Initialization

The historian shall automatically begin to acquire tag data immediately after a tag configuration has been committed to the database. Adding a single or multiple tags to an existing historian database shall not affect the data acquisition of previously defined tag names.

Client connections will not be affected during reconfiguration due to dynamic configuration. Additionally, there will be no loss of data for tags where data acquisition configuration is not changed. Tags that require a change in data acquisition configuration will obviously lose data during the period of their re-initialization.

11.7.2 Manual Data, Out-of-Sequence Data, and Superseded Data

The historian shall support Manual Data, Out-of-Sequence Data, and Superseded Data. Manually entered data such as Lab Data and Out-of-Sequence Data such as batched history data from a Remote Terminal Unit (RTU) shall be treated by the retrieval engine as if the data were stored automatically. Any historized data may be superseded by manually inserting the correct data value and a flag denoting that the previous data has been superseded. The original data shall not and cannot be modified or destroyed. An SQL client tool may request original data, superseded data, or both. Manual Data, Out-of-Sequence Data, and Superseded Data shall be inserted into the historian via an SQL Insert statement, or in bulk via Comma Separated Variable (CSV) file. Only users with proper login credentials shall be allowed to manually insert or modify data.

Manual, or corrected data is recognized as "versioned data" leaving the original data intact. Queries shall be executed against the historical data using a switch to return "Original" data results.

11.7.3 Data Quality

All stored data shall contain data quality attributes. The primary Data Quality attribute shall reflect Data Quality as defined in OLE for Process Control (OPC). Additional quality attributes shall be used for initial data (startup flag) and superseded data.

11.7.4 Event System Configuration

The data historian shall contain an event sub-system to monitor, record, and or respond to process or system events and to trigger some type of action when the event is detected.

The event system shall detect an event occurrence using pre-defined and configurable criteria, historically log when an event occurs and trigger designated configurable event actions based on the event detection. Event attributes shall be logged to the database and will include the date, time that the event occurred, and the event criteria that were satisfied.

11.7.5 Event Configuration

Using a point and click approach, the system shall allow users to create/define event tag names and associate event tag names to event detectors and the resulting actions.

The user shall be able to insert a time delay in milliseconds before the event action is triggered and establish the priority of the event as normal or critical. It shall be possible to detect an event based on scheduled time interval, a specific analog value crossing a threshold or a discrete value going from 0 to 1 (leading edge), 1 to 0 (trailing edge), or both. An event editor shall be provided to support complex SQL event detectors and event actions.

11.7.6 Event action

The data historian event detectors shall determine that an event has occurred and trigger an associated action. Event detectors shall scan for events at the user-defined rate for each event. The user shall be able to select from any one of four actions when an event is detected. Event actions shall include:

- Execution of an SQL statement to perform an SQL query on the database
- Taking a snapshot and recording the time stamp and values of one or more selected analog or discrete tags
- Sending a Microsoft Exchange e-mail message to designated recipients
- Change the cyclic time and/or value delta storage rate for one or more analog tag names.

The ability to modify the storage rate based on an event allows fast logging of data to gather more valuable process data to assist in troubleshooting.

11.7.7 Simple Unit Conversion within historian client

Choose engineering units to automatically convert between, with 600 predefined catalog units and the ability of defining custom units.

11.7.8 Historian client Excel Add-in (64-bit)

- Batch-oriented summary statistics: Build reports with the capabilities of slicing by using an event ID, batch number, or conditional states to get summary statistics
- Dynamically refresh custom functions
- Function signatures for better reporting

11.7.9 Browser-based Historical Alarm Grid

Alarm grid in historian Client that should be filtered based on the model view or the HMI alarm groups.

11.7.10 Data Replication Across Satellite Connections

Support for data replication between historians across high-latency networks (multi-second ping times across network).

11.7.11 Tag Extended Property Administration

SMC supports adding/editing/deleting tag extended properties.

11.8 Historian Interface to Other Relational Databases

The data historian shall utilize Microsoft's Data Transformation Services to simplify the transfer of historical process data with other SQL Server databases like Microsoft SQL Server or Oracle. The historian database shall include an OLE DB Provider (Object Linking and Embedding for Databases) so that any other 3rd party SQL client should be able access the real-time or historical process data from historian.

11.9 Disk Storage Management

The data historian shall not require specialized tools for disk storage management. It shall be possible to archive and retrieve historical data files using standard Windows® copy techniques. It shall be possible to retrieve select portions of archived data without retrieving all archived data. Retrieval of the archive data shall automatically place this data on-line and available for retrieval by the data historian.

Additionally, the data historian shall approach zero-administration as nearly as possible. The data historian shall provide for a mechanism whereby current files on a disk drive that is nearly full will automatically be moved to a secondary device. The files and available space on the secondary drive shall be monitored as well such that when a user-defined threshold is reached, the oldest files may be automatically deleted to preserve the integrity of the system. Historical files shall never be deleted from the primary storage device if an appropriate secondary device is configured.

12.0 Historian Data Analysis Clients

A comprehensive suite of Data Analysis Client software shall be available from the same vendor to enable Historical data analysis.

12.1 Standard Query Language Interface

The system shall support the use of standard SQL to perform queries against the historian database.

The SQL interface provided shall include extensions pertaining to the retrieval of time-based information, including the ability to retrieve information based on rules pertaining to the known characteristics of the information. Such retrieval modes shall include:

- Average
- Best Fit
- Counter
- Cyclic
- Delta
- Full
- Integral
- Interpolated
- Maximum
- Minimum
- Round Trip
- Slope
- Time In State
- Time-weighted average
- Value State

12.2 Historical Trend Analysis Tool

A stand-alone client tool shall be included that allows users to view any or all the Historized tag names in either a trend chart or tabular format. The client tool shall have a user interface that allows for easy selection of tag names using a Windows Explorer-like browser with a search filter to quickly find tag names in a data historian with thousands of points.

The user shall be able to create folders for selected groups of tag names and plot them individually or in groups by dragging them into the trend area. The user shall be able to save trend files for recall at a later time. It shall be possible for the user to switch from a real time view to the historical viewing mode using a simple check box.

The user shall be able to toggle from viewing trends either in a superimposed or stacked mode. In the superimposed mode, all trends overlap and are in a single scale range based on the largest vertical scale range in the group. In the stacked mode, each trend has its own vertical scale range. Trend plots shall automatically be scaled based on the widest vertical range of the tag name or optimized based on the maximum and minimum range within the selected time period.

The user shall be able to plot historical data for any tag name or groups of tag names in the database based any user-selected start and stop time. Two hairline cursors may be turned on and dragged across the trend area to provide the user with the exact value for each trended tag name at the point of intersection. The time span and the value between the cursors shall also be displayed. It shall be possible to overlay data from different start/end times to compare the performance of equipment / compare the process for different time intervals. It shall be possible to overlay 'live' trends onto history traces to compare performance.

The trend tool shall display statistical data for each trended analog tag name within the time period selected. Statistical values shall include the minimum, maximum, average, and standard deviation. Icons or menu pull down commands shall be available for analyzing the data such as horizontal, vertical or rubber band zooming pan left or right and zoom between the hairline cursors.

It shall be possible for the user to create text annotations anywhere on the trend. These annotations shall be visible from other workstations on the network with the same trend tool. It shall be possible to export the data in the trend area into a CSV file. Printing of the trends with all statistical data shall be supported.

The tool shall be available as ActiveX, .NET WinForms and .NET WPF components for use with acceptable containers, such as the HMI. A web version of the tool with appropriate functionality shall also be available.

12.3 Historical Query Generator Tool

A stand-alone query client shall be included to allow the user to execute SQL queries that returns a result set from any SQL server database into a tabular data grid. The user shall be able to generate the necessary SQL queries without actual knowledge of SQL by selecting tags, conditions and retrieval modes within the query generator tool interface.

The query generator tool shall support multiple data server sources for simultaneous display of the data.

The tool shall be available as ActiveX and .NET components for use with acceptable containers, such as the HMI.

12.4 6.4. Microsoft Excel Interface

The data analysis software shall provide an interface which enables reports to be constructed from within Microsoft Excel.

Users shall be able to easily select tag names and historical values from the real time or historical database via a browser and then utilize them in a standard Microsoft Excel spreadsheet for reporting or presentation to management. The selection of tag names shall be accomplished by use of drag and drop or point and click commands. It shall not be required to write any macros to retrieve the data. The tag values selected can be output to specific cells in the spreadsheet and

processed as number data types. The user shall be able to select historical data for the most recent values or go back and select any start or stop time as far back as the data is available. The historical data shall be recalled at the granularity that it was stored or in a selected number of data points over a period of time.

The user shall be able to retrieve raw historical data or summarized data such as the minimum, maximum or average over a predetermined time period. Updates to the current values once they are in the spreadsheet shall be refreshed with a single click of the mouse. The quality of the data shall be analyzed and displayed. The user shall be able to select if poor quality data is to be displayed or replaced with an interpolated value. The user shall be able to specify relative or absolute value choices.

12.5 Microsoft Word Interface

The data analysis software shall provide an interface which enables reports to be constructed from within Microsoft Word.

Users shall be able to easily select tag names from the real time or historical database via a browser and then utilize them in a standard Microsoft Word document for reporting.

The selection of tag names shall be accomplished by use of drag and drop or point and click commands. It shall not be required to write any macros to retrieve the data. The tag names selected shall be output to specific values or arrays using standard Word tables. The user shall have the choice of selecting real time, absolute, relative or configurable base dates and times. The user shall be able to retrieve raw historical data, summarized data such as the minimum, maximum or average over a predetermined time period or data stored in the historian event system. The user shall be able to select if poor quality data is to be displayed or replaced with an interpolated value.

12.6 Cloud Management

The software should have seamless integration to Cloud Platform and allows on-premise application to seamlessly securely connect. The cloud platform provide capability to store, visualized real-time data, build dashboards, HMI and add alerts. System shall collect and aggregate data across your Operations including edge devices and visualize them in the cloud, where AI and Machine Learning shall be added and enable rapid decision making. This should work as SaaS offering and also support for externally detected alarms (in the control system instead of using built-in SCADA).

Provision software via secure cloud includes:

- Develop your applications.
- Simultaneously monitor multiple sites from any geographic location.
- Foster collaborative concurrent development across teams.
- Save library of Common Graphics across teams enabling enterprise standardization.
- Share and collaborate on graphics between projects and developers using common cloud storage.

13.0 HTML5 Web Clients

The system shall offer web capabilities for aggregating and presenting the full spectrum of SCADA collected and generated content including real-time and historical process and performance data, graphics, trends, alarms and reports with HTML5 web client. It should allow read and write capability. It shall also allow mobile access.

13.1 Web Solution for Reporting

The web solution shall also provide ability to incorporate other additional non-SCADA data sources.

13.1.1 HTML5 Reporting Client Tool

A client tool shall be included that allows users to generate reports on any tag names. The client tool shall have a user interface that allows selection of tag names and previously created reports and report formats. The report tool shall allow users to set up, modify, and generate reports that present report data professional and reliable.

Types of information supported shall include historical and current data values, tag name configuration information, graphs, statistics, annotations, event and summary information and results from a query. Reports shall be generated based on data stored over a specified time period in minutes, hours, days, weeks or months. It shall also be possible to generate reports on demand by the user at runtime. Finished reports shall be formatted in HTML so that they can be viewed with a Web Browser, stored on the server disk or sent to a printer.

13.1.2 Report Security

It shall be possible to configure reports to only be visible and accessible to certain users or groups.

13.2 HMI Objects/Visualization

- The system must support the ability to show HMI screens within a standard browser interface.
- The system must support the ability to show graphical symbols, which are part of application objects in standard browser.
- The system must support the graphic layout to be used in Cloud Infrastructure in order to reduce the engineering efforts.
- Link a layout residing in the Graphic Toolbox to an asset or template and have it participate in auto-fill activities when the associated asset comes into focus.
- A layout's owning object shall be set and it will auto-fill based on the owning object's content.
- External content objects support expressions enabling the dynamic determination of content at runtime.

14.0 Software Warranty, Maintenance and Support

The software vendor shall provide software maintenance and support program to ensure that the user receives full benefit of the software for the duration of its life cycle. The program shall provide for basic warranty coverage and include an extended warranty for priority support and software upgrades as they are released. Telephone support shall be available through a toll-free number during normal business hours. Support shall also be available via fax, email or through a technical support website.

14.1 Warranty Support

The software vendor shall warrant the products for a period of 90 days after delivery. During the warranty period, the vendor shall offer free technical telephone support during normal business hours through a toll-free number. All software defects shall be resolved in a timely manner.

14.2 Extended Support and Software Maintenance

After the 90-day warranty period, the user may continue to receive technical support via fax, email or access the technical support website. In order to ensure that the user always has access to the latest software releases, long-term warranty and technical support, the vendor shall offer an extended support program for a fixed annual fee.

14.2.1 Software Upgrades

The extended support program shall entitle the user to receive the latest SCADA system software releases and version upgrades, as they become available. In order to ensure quality support for all users, all software licenses at the site must be maintained at the same version level.

14.2.2 Operating System Patch Support

The vendor shall test and support operating system patches which are periodically released by Microsoft. The vendor shall have a defined policy of support for such security patches.

The Software Vendor shall provide a Security portal that also enables our users to review the patches, any potential risk that may impose on our systems and when the patches have been cleared for installation without risk.

14.2.3 Telephone Support

The extended support program shall include telephone support during normal local business hours. A technical support engineer who has been certified by the software vendor based on a certified support testing program shall provide telephone support. During normal business hours, we shall provide unlimited telephone support by a support engineer. A voice-mail tech support system shall not be acceptable.

14.2.4 Email Support

The extended support program shall include e-mail support within one business day at a higher priority than non-warranty support users and will forward them to the nearest certified technical support center. Electronic support shall also include expanded access to advanced services on our technical services web page. The extended support program shall include real-time access to current and past issues in a call tracking database, as well as the ability to create new issues, which shall be immediately assigned to a technical support engineer for resolution.

14.2.5 Electronic File Download

The extended support program shall include access to a secure web site for electronic file downloads. New software releases, service packs, patch fixes; updated I/O Servers and other support files shall be available for support users from the secure web site.

14.2.6 Web Based Support

The software vendor shall have a development and support-oriented web site which provides technical information on the SCADA software, best practices for implementation of the SCADA software and user forums.

14.2.7 Newsletter and Technical Support CD

The software vendor shall provide a newsletter and a technical support CD with tech notes to all users on the extended support program a minimum of two times per year. The technical support CD shall include a comprehensive summary of technical notes, technical alerts, applications, application utilities, diagnostic utilities, controls, drivers, scripts, script functions, wizards and helpful hints that can streamline application development.

The Software Vendor shall also provide an online, updated and fully searchable website with same content.

14.3 Software backward compatibility

The software vendor shall have a track record of no less than 10 years of software backward compatibility and continuous migration path in order to protect engineering investment. Old applications shall be able to be easily migrated to the newest versions of the software without engineering modifications.

14.4 Cybersecurity

The vendor must have published security guidelines to assist in securing the entire SCADA system. The information pertaining to security implementation and updates shall be available on software vendor website.

15.0 Document Control

15.1 **Approval:** GLOBAL_FAC_GFTT_APPR

15.2 Notification:

GLOBAL_FAC_MANAGERS
GLOBAL_FAC_COMMUNICATIONS
GLOBAL_FAC_CONSTRUCTION
GLOBAL_FAC_NOTIFY
FCG_F4_FAC_NOTIFY
FCG_F6_FAC_NOTIFY
FCG_F10_FAC_NOTIFY
FCG_F11_FAC_NOTIFY
FCG_F15_FAC_NOTIFY
FCG_F16_FAC_NOTIFY
FCG_MMY_FAC_NOTIFY
FCG_MMP_FAC_NOTIFY
FCG_MSB_FAC_NOTIFY
FCG_MTB_FAC_NOTIFY
FCG_MXA_FAC_NOTIFY

15.3 Final Viewer Access

All Micron sites Facilities Team Members

15.4 Retention

Adherence to the requirements specified in the
[**Global Facilities - Document Control Procedures**](#)

15.5 Review

Biennially (two years)

16.0 Revision History

Revision #	Section Changed	Details of Changes	Date
0	NA	New document	03/21/22

Appendix A: Deviation List

Any deviation from this specification shall be submitted for approval during vendor evaluation. Deviation list format shall be the following:

Site: _____

No	Specification Section and Item	Deviation Proposal	Scope Impact (Benefit, Cost, Schedule)	Remark	Approval: 1. Site 2. GFTT